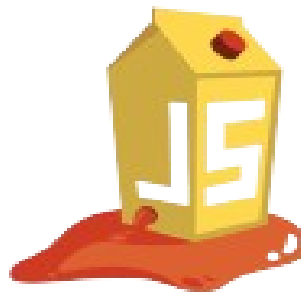


Vulnerability Assessment and Penetration Testing (VAPT) Report

Prepared by **Lvl0**
Prepared for: owasp_juice_shop

January | 08 | 2025



No warranties, express or implied, are provided regarding the accuracy, reliability, completeness, or correctness of this report. This document is presented "as is," and the authors shall not be held liable for any errors, omissions, or inaccuracies. While efforts have been made to ensure the validity of the information, we do not guarantee that all errors have been identified or corrected.

This report is intended for academic purposes only and does not constitute professional or legal advice. Any reliance on the information contained herein is at the reader's own discretion. The inclusion of any company names, products, or technologies does not imply endorsement or affiliation.

©2025 JCE. All rights reserved. No part of this document may be reproduced, copied, or modified without proper citation or prior written permission from the authors. This document is intended for educational use only and should not be shared with third parties without authorization.

EXECUTIVE SUMMARY	1
TEST SCOPE	1
RESULTS	1
Identified Vulnerabilities and Recommended Remediations	2
RECOMMENDATIONS	3
TESTING APPROACH	5
OVERVIEW	5
DISCOVERY & RECONNAISSANCE	6
Significant Vulnerability Details:	7
1. Cloud Metadata Potentially Exposed	12
2. Content Security Policy (CSP) Header Not Set	12
3. Cross-Domain Misconfiguration	13
4. Hidden File Found	13
5. Missing Anti-Clickjacking Header	14
6. Session ID in URL Rewrite	14
7. Vulnerable JavaScript Library	15
8. Cross-Domain JavaScript Source File Inclusion	15
9. Private IP Disclosure	16
10. Timestamp Disclosure - Unix	16
11. X-Content-Type-Options Header Missing	16
12. Information Disclosure – Suspicious Comments	17
13. Modern Web Application	17
14. Retrieved from Cache	18

EXECUTIVE SUMMARY

This report presents a comprehensive security assessment of OWASP_JUICE_SHOP conducted to identify existing vulnerabilities and evaluate the overall security risks associated with its IT infrastructure and technologies. The assessment incorporated penetration testing and social engineering techniques to analyze potential threats and determine the organization's security posture. The findings aim to provide valuable insights for strengthening cybersecurity measures and mitigating risks within the corporate environment.

TEST SCOPE

The purpose of the engagement is to utilize assessment techniques in order to identify and validate

potential vulnerabilities across all systems within scope.

SCOPE: 127.0.0.1:3000

RESULTS

The table below includes the scope of the tests performed, as well as the overall results of penetration testing these environments.

No high impact vulnerabilities were found. But a significant number of medium impact vulnerabilities were found.

HIGH SEVERITY	MEDIUM SEVERITY	LOW SEVERITY	INFORMATIONAL
1	6	4	3

Below is a summary of all the vulnerability found during the assessment and their remediations:

Identified Vulnerabilities and Recommended Remediations

Vulnerability	Severity Level	Recommended Remediation
Cloud Metadata Potentially Exposed	High	Restrict access to cloud metadata endpoints using firewall rules and IAM policies to prevent unauthorized access.
Content Security Policy (CSP) Header Not Set	Medium	Implement a Content Security Policy (CSP) header to prevent cross-site scripting (XSS) and data injection attacks.
Cross-Domain Misconfiguration	Medium	Restrict cross-domain access by properly configuring CORS policies to allow only trusted origins.
Hidden File Found	Medium	Remove or restrict access to hidden files that may expose sensitive information. Conduct periodic security reviews.
Missing Anti-Clickjacking Header	Medium	Implement the X-Frame-Options header with "DENY" or "SAMEORIGIN" to protect against clickjacking attacks.
Session ID in URL Rewrite	Medium	Store session IDs in secure HTTP-only cookies instead of passing them through URLs.

Vulnerable JavaScript Library		Medium	Update JavaScript libraries to the latest secure versions to mitigate known vulnerabilities.
Cross-Domain JavaScript Source File Inclusion		Low	Ensure JavaScript files are loaded only from trusted sources and cannot be controlled by end users.
Private IP Disclosure		Low	Prevent private IP exposure by reviewing server responses and filtering sensitive data.
Timestamp Disclosure - Unix		Low	Confirm that the timestamp data is not sensitive and cannot be used to infer exploitable patterns.
X-Content-Type-Options Header Missing		Low	Set the X-Content-Type-Options header to "nosniff" to prevent MIME-type sniffing attacks.
Information Disclosure - Suspicious Comments		Informational	Remove all comments that may leak sensitive information and fix underlying issues they reference.
Modern Web Application		Informational	No immediate action required, but ensure best security practices are maintained.
Retrieved from Cache		Informational	Implement cache-control headers to prevent sensitive information from being stored in browser caches.

RECOMMENDATIONS

SampleCorp – Penetration Test Report

Based on the available findings from the security assessment, the following recommendations are provided to enhance the security posture of OWASP Juice Shop:

1. **Prioritize Medium-Risk Findings**

- Identified medium-risk vulnerabilities should be addressed first to mitigate potential security threats.
- Implement secure coding practices and apply patches to vulnerable components.

2. **Restrict Access to Sensitive Data**

- Ensure that sensitive data is not accessible in an unauthenticated manner.
- Implement **IP address whitelisting** to restrict unauthorized access to critical resources.

3. **Comprehensive Security Assessment**

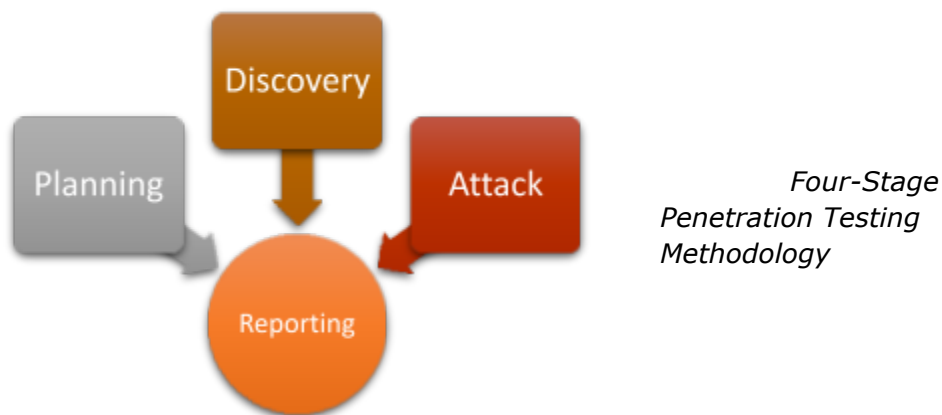
- The findings from this scan should not be considered a definitive measure of the overall security posture.
 - Additional security assessments, including:
 - **Policy Review** – Evaluating security policies to align with best practices.
 - **Internal Security Controls & Procedures** – Assessing security frameworks and processes.
 - **Red Teaming & Penetration Testing** – Conducting hands-on security testing to simulate real-world attack scenarios.
-

TESTING APPROACH

OVERVIEW

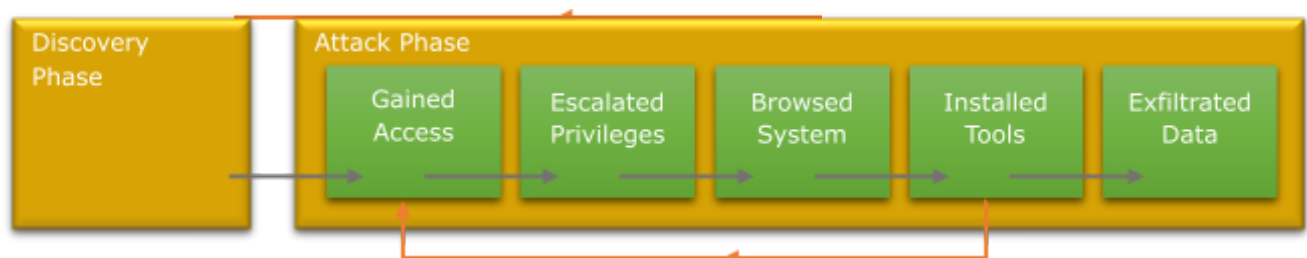
All testing was executed in several related phases.

1. In the planning phase, the rules of engagement were identified, scope of testing and test windows were agreed upon, and testing goals were set.
2. The discovery phase included automated vulnerability scanning along with manual testing to explore and understand the testing target and any vulnerabilities that could be detected by automated tools.
3. The attack phase comprised efforts to exploit any vulnerabilities detected, and to synthesize knowledge gained about the website, its technology, its users and its function into an escalation of privilege beyond that intended by the customer.
4. The final phase recorded all findings in a manner that supports risk assessment and remediation by the customer. This included the writing of this report.



Additionally, the attack phase comprised several distinct steps, executed iteratively as information was discovered.

1. Gained access to the system or environment in a way that was not intended.
2. Escalated privileges to move from regular or anonymous user to a more privileged position.
3. Browsed to explore the newly accessed environment and identify useful assets and data.
4. Deployed tools to attack further from the newly gained vantage point.
5. Exfiltrated data.

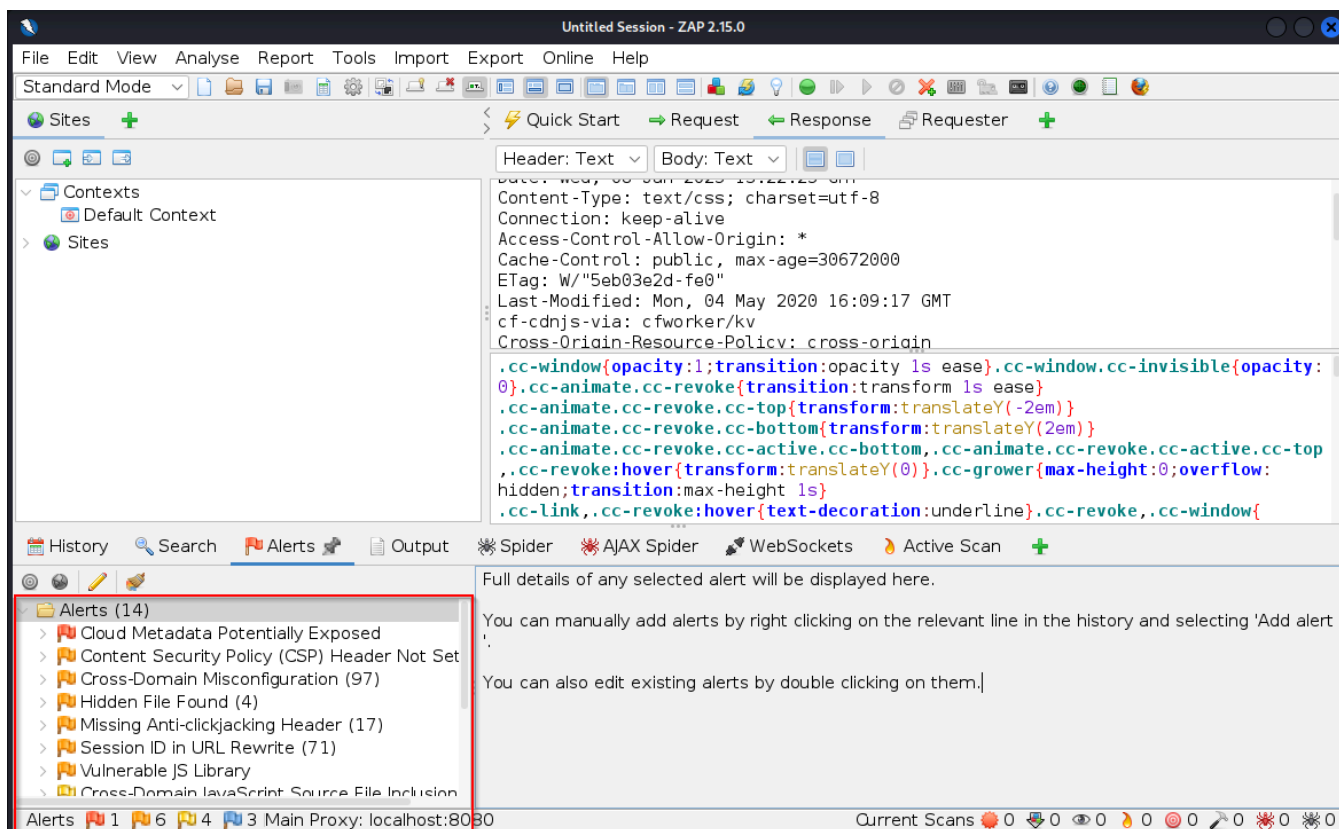


DISCOVERY & RECONNAISSANCE

As the initial phase of this security assessment, reconnaissance and discovery were conducted to gain insights into the OWASP Juice Shop environment. This process involved:

Network and Application Scanning

- Identifying exposed services, open ports, and potential attack surfaces.
- Utilizing tools like **Nmap**, **OWASP ZAP**, and **Burp Suite** to detect vulnerabilities.



Dirbuster:

File Name	Size
/	200
api.php	500
restaurants.php	500
profile	500
public	200
redirect	500
ajax	200
ftp	200
api	500
rest.php	500
video	200

Significant Vulnerability Details:

1. Cloud Metadata Potentially Exposed

Description: The cloud metadata service is accessible from within the application, allowing attackers to retrieve sensitive instance credentials.

Impact: An attacker can exploit this to gain access to cloud services, extract IAM credentials, and escalate privileges.

Severity: High

CVSS v3.1 Score: 9.0 (Critical Severity)

CVE:

- CVE-2019-7609 – Kibana SSRF vulnerability allowing metadata exposure.
- CVE-2019-5418 – Arbitrary file read vulnerability leading to metadata exposure.

Proof of Concept (PoC):

```
curl -H "Metadata: true" http://169.254.169.254/latest/meta-data/
```

```

<!--
~ Copyright (c) 2014-2024 Bjoern Kimminich & the OWASP Juice Shop contributors.
~ SPDX-License-Identifier: MIT
--><!DOCTYPE html><html lang="en"><head>
<meta charset="utf-8">
<title>OWASP Juice Shop</title>
<meta name="description" content="Probably the most modern and sophisticated insecure web application">
<meta name="viewport" content="width=device-width, initial-scale=1">
<link id="favicon" rel="icon" type="image/x-icon" href="assets/public/favicon_js.ico">
<link rel="stylesheet" type="text/css" href="//cdnjs.cloudflare.com/ajax/libs/cookieconsent2/3.1.0/cookieconsent.min.css">
<script src="//cdnjs.cloudflare.com/ajax/libs/cookieconsent2/3.1.0/cookieconsent.min.js"></script>
<script src="//cdnjs.cloudflare.com/ajax/libs/jquery/2.2.4/jquery.min.js"></script>
<script>
  window.addEventListener("load", function(){
    window.cookieconsent.initialise({
      "palette": {
        "popup": { "background": "var(--theme-primary)", "text": "var(--theme-text)" },
        "button": { "background": "var(--theme-accent)", "text": "var(--theme-text)" }
      },
      "theme": "classic",
      "position": "bottom-right",
      "content": { "message": "This website uses fruit cookies to ensure you get the juiciest tracking experience.", "dismiss": "Me want it!", "link": "But me wait!", "href": "https://www.youtube.com/watch?v=9PnbKL3wuH4" }
    });
  });
</script>
<style>.bluegrey-lightgreen-theme{--theme-primary:#546e7a;--theme-primary-lighter:#607e8c;--theme-primary-light:#698998;--theme-primary-darker:#485e68;--theme-primary-dark:#3f535c;--theme-primary-fade-10:rgba(84, 110, 122, .9);--theme-primary-fade-20:rgba(84, 110, 122, .8);--theme-primary-fade-30:rgba(84, 110, 122, .7);--theme-primary-fade-40:rgba(84, 110, 122, .6);--theme-primary-fade-50:rgba(84, 110, 122, .5);--theme-accent:#689f38;--theme-accent-lighter:#77b640;--theme-accent-light:#81bf4b;--theme-accent-darker:#598830;--theme-accent-dark:#4f792b;--theme-accent-fade-10:rgba(104, 159, 56, .9);--theme-accent-fade-20:rgba(104, 159, 56, .8);--theme-accent-fade-30:rgba(104, 159, 56, .7);--theme-accent-fade-40:rgba(104, 159, 56, .6);--theme-accent-fade-50:rgba(104, 159, 56, .5);--theme-warn:#ff5722;--theme-warn-lighter:#ff6e41;--theme-warn-light:#ff7e55;--theme-warn-darker:#ff4003;--theme-warn-dark:#ee3900;--theme-warn-fade-10:rgba(255, 87, 34, .9);--theme-warn-fade-20:rgba(255, 87, 34, .8);--theme-warn-fade-30:rgba(255, 87, 34, .7);--theme-warn-fade-40:rgba(255, 87, 34, .6);--theme-warn-fade-50:rgba(255, 87, 34, .5);--theme-text:white;--theme-text-lighter:white;--theme-text-light:white;--theme-text-darker:#e6e6e6;--theme-

```

Mitigations:

Restrict access to cloud metadata endpoints using firewall rules and IAM policies to prevent unauthorized access.

2.Content security policy not set

A missing **Content Security Policy (CSP)** header exposes web applications to security risks, particularly **Cross-Site Scripting (XSS)** and **Clickjacking** attacks.

Severity: Medium

CVSS Score: 7.5

CVE:

CVE-2018-12037

- Description: Lack of CSP and improper sanitization led to stored XSS in certain web applications.

SampleCorp – Penetration Test Report

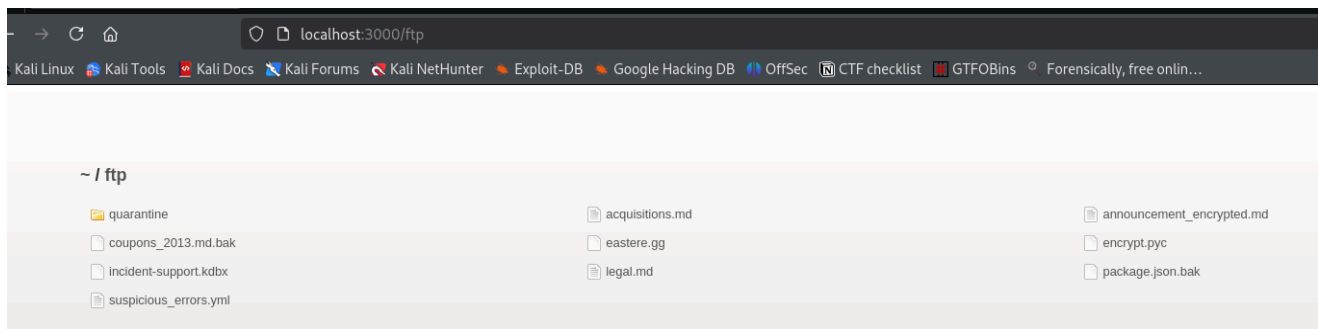
CVE-2020-11022

- Description: jQuery improperly sanitized input, leading to potential XSS, which could have been mitigated with a proper CSP.

CVE-2016-4067

- Description: A web application failed to set security headers like CSP, allowing attackers to exploit stored XSS.

POC:



3. Cross Domain misconfiguration

Cross-domain misconfigurations occur when web applications improperly set **CORS (Cross-Origin Resource Sharing)** policies, allowing unauthorized access to sensitive resources from other domains. This can lead to **data theft, unauthorized API access, and account takeover** in some cases.

Severity: Medium

CVSS Score: 7.3

CVE:

CVE-2021-44790

- Description: Improperly configured CORS in Apache HTTP Server allowed unauthorized requests, leading to potential remote code execution.

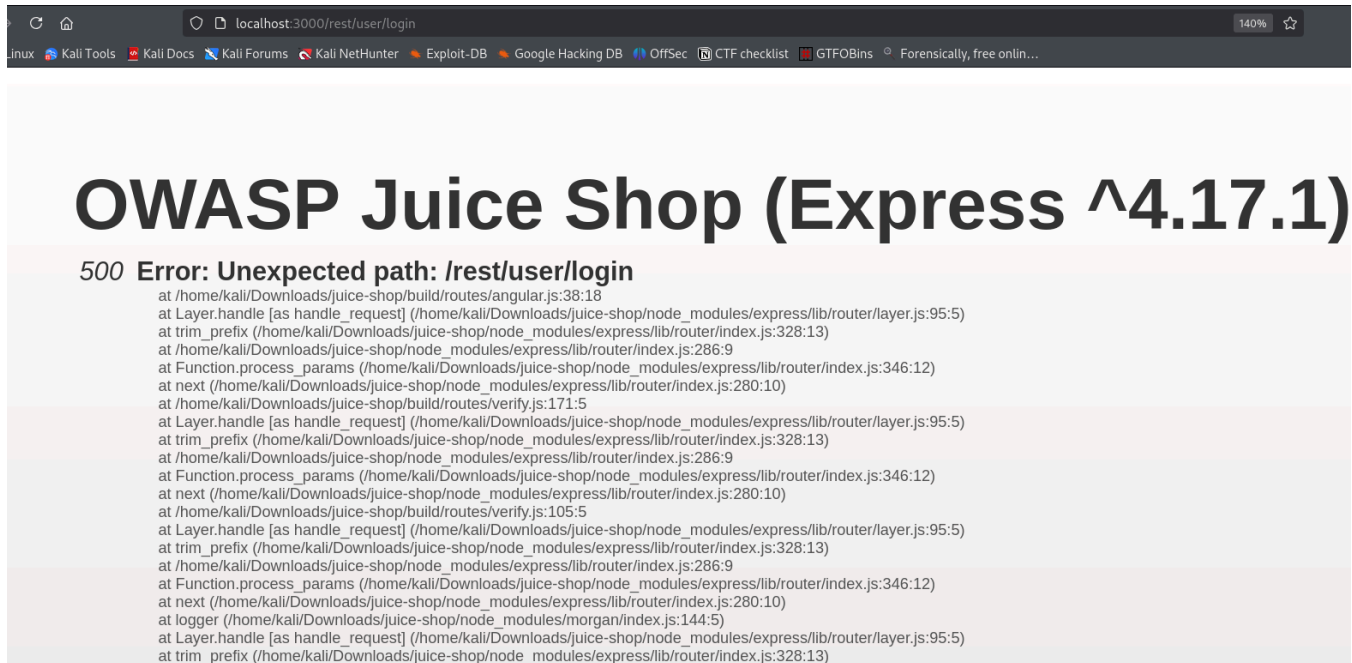
CVE-2020-35592

SampleCorp – Penetration Test Report

- Description: Weak CORS policy in certain web applications led to exposure of sensitive information across different origins.

CVE-2018-3760

- Description: A misconfigured CORS header in a Node.js application allowed unauthorized cross-domain requests.



Mitigations:

Implement a **Content Security Policy (CSP)** header to prevent cross-site scripting (XSS) and data injection attacks.

4. Missing Anti-clickjacking Header:

When a web application lacks **anti-clickjacking protection**, attackers can embed the website inside an **invisible iframe** and trick users into performing unintended actions, such as clicking on malicious buttons, changing settings, or authorizing transactions.

Severity: Medium

CVSS Score: 6.8

CVE:

CVE-2018-19933

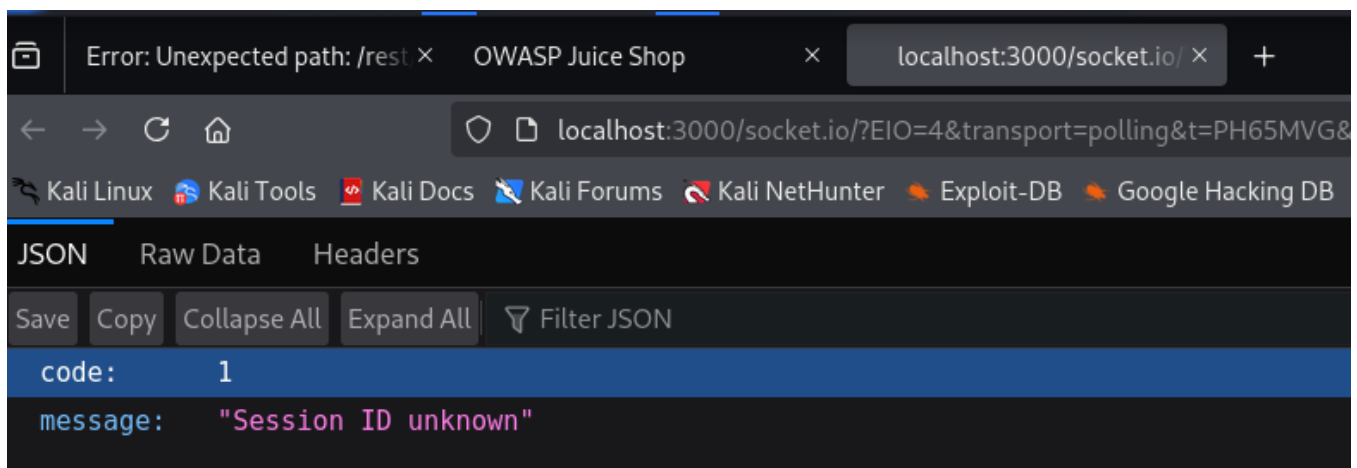
- **Description:** A missing **X-Frame-Options** header in a web application allowed clickjacking attacks, exposing users to UI redressing techniques.

CVE-2016-7191

- **Description:** Internet Explorer did not properly enforce clickjacking protections, allowing attackers to exploit frame-based interactions.

CVE-2019-9853

- **Description:** An application was vulnerable to clickjacking because it failed to set **X-Frame-Options** or **Content-Security-Policy: frame-ancestors** headers.



Remediation: Implement the **X-Frame-Options** header with "DENY" or "SAMEORIGIN" to protect against clickjacking attacks.

Here's a **detailed list of remediations** for each identified vulnerability:

1. Cloud Metadata Potentially Exposed

- **Remediation:**
 - **Restrict Access to Cloud Metadata Endpoints:**
 - Cloud metadata endpoints, often used for instance management and configuration, can be accessed by anyone on the network unless properly restricted.
 - Use firewall rules to block unauthorized requests to metadata services.
 - Implement **Identity and Access Management (IAM)** policies to limit access to cloud metadata to authorized users and services only.
 - Ensure metadata services are isolated and protected from external access and leaks.
 - Configure **IAM roles** to limit which entities can access metadata endpoints in cloud environments like AWS, Azure, and Google Cloud.
-

2. Content Security Policy (CSP) Header Not Set

- **Remediation:**
 - **Implement Content Security Policy (CSP):**
 - **CSP** is a powerful browser security feature that helps to prevent cross-site scripting (XSS) and data injection attacks by specifying which sources of content the browser should allow.

Define a strict CSP header such as:

```
Content-Security-Policy: default-src 'self'; script-src 'self'  
'https://trusted-cdn.com'; style-src 'self' 'https://trusted-cdn.com';
```

■

- **Whitelist trusted content sources** to limit external resources and scripts.
 - Regularly update and audit your CSP to ensure that malicious content is not allowed by mistake.
-

3. Cross-Domain Misconfiguration

- **Remediation:**
 - **Configure Cross-Origin Resource Sharing (CORS) Policies Properly:**
 - CORS allows resources to be requested from a domain other than the one that served the original web page.
Misconfigurations in CORS settings can expose your site to unauthorized access or data leakage.

Configure **CORS policies** to allow only trusted domains to make cross-origin requests. Example:

Access-Control-Allow-Origin: https://trusted-origin.com

- - Avoid using wildcard (*) for **Access-Control-Allow-Origin**.
Always specify the exact origin to prevent unauthorized access.
 - Ensure only authorized methods (e.g., GET, POST) and headers (e.g., Content-Type) are permitted.
-

4. Hidden File Found

- **Remediation:**
 - **Remove or Restrict Access to Sensitive Files:**
 - Hidden files or files with sensitive data (e.g., **.git**, **.env**, **.svn**) should never be exposed on the web.
 - Regularly review the server's file system and remove any unnecessary or hidden files that could provide an attacker with

insights into your environment.

- Use proper **file permission settings** to ensure that sensitive files are only accessible by authorized users.
 - Implement **directory listing protection** to prevent attackers from discovering hidden files through web server misconfigurations.
-

5. Missing Anti-Clickjacking Header

- **Remediation:**

- **Implement Anti-Clickjacking Protection:**

- **Clickjacking** is an attack where an attacker tricks a user into clicking on something different from what the user perceives, often by embedding malicious content in an invisible iframe.

To mitigate this, implement the **X-Frame-Options** header:

X-Frame-Options: DENY

- - This will block your content from being embedded in iframes on other websites.
 - Alternatively, use **SAMEORIGIN** to allow your site to be embedded only within frames of the same origin.
 - In some cases, use **Content Security Policy (CSP)** with the **frame-ancestors** directive as an additional layer of protection.
-

6. Session ID in URL Rewrite

- **Remediation:**

- **Avoid Passing Session ID in URLs:**

- Storing session IDs in the URL (e.g., **www.example.com?sessionid=abc123**) is insecure, as URLs can be logged, cached, and exposed in browser history.

- Instead, store session IDs in **secure, HttpOnly, SameSite cookies** to prevent client-side access.

Example cookie configuration:

Set-Cookie: sessionid=abc123; HttpOnly; Secure; SameSite=Strict

- - Ensure session data is never exposed in URLs or query parameters.
-

7. Vulnerable JavaScript Library

- **Remediation:**
 - **Update JavaScript Libraries to Secure Versions:**
 - Using outdated JavaScript libraries can introduce known security vulnerabilities.
 - Regularly check for updates and security patches for third-party libraries and frameworks (e.g., jQuery, React, Angular).
 - Use a **dependency management tool** like **npm audit** or **Snyk** to identify and address vulnerable packages.
 - Where possible, use **Subresource Integrity (SRI)** to verify that the content has not been tampered with when served from third-party sources.
-

8. Cross-Domain JavaScript Source File Inclusion

- **Remediation:**
 - **Restrict the Loading of JavaScript from Untrusted Sources:**
 - Ensure that all JavaScript files are sourced only from trusted and controlled locations.
 - Never allow end users to control or define where JavaScript files are loaded from.
 - Validate and sanitize all inputs that specify script source

- Implement CSP to restrict script loading to trusted sources only.
-

9. Private IP Disclosure

- **Remediation:**
 - **Prevent Disclosure of Private IPs:**
 - Avoid exposing internal IP addresses in error messages, headers, or through other mechanisms accessible to external users.
 - Implement proper filtering to ensure that private IPs are not leaked in HTTP responses or other communication channels.
 - Use **IP address filtering** to ensure that requests from unauthorized networks do not reveal internal infrastructure details.
-

10. Timestamp Disclosure - Unix

- **Remediation:**
 - **Ensure Timestamp Data is Not Sensitive:**
 - If timestamp data is exposed, confirm it is not sensitive or does not correlate with other information that could lead to exploitation.
 - Avoid exposing full timestamps in user-visible areas, especially in combination with other data that could help identify attack vectors (e.g., logs).
 - If necessary, obfuscate or limit access to timestamp data that may be considered sensitive.
-

11. X-Content-Type-Options Header Missing

- **Remediation:**
 - **Implement the X-Content-Type-Options Header:**

- This header helps prevent the browser from performing MIME type sniffing, which could lead to **cross-site scripting** or other content-type attacks.

Add the following header to your server responses:

X-Content-Type-Options: nosniff

■

12. Information Disclosure – Suspicious Comments

- **Remediation:**
 - **Remove Sensitive Information from Comments:**
 - Check for comments in the code or documentation that may leak sensitive information (e.g., internal IP addresses, credentials, environment details).
 - Remove any comments that provide hints to potential attackers, or that disclose vulnerabilities or system details.
 - Regularly review code for suspicious comments and audit older versions of code repositories for hidden information.

13. Modern Web Application

- **Remediation:**
 - **Maintain Best Security Practices:**
 - While this is informational, continue to stay updated on security best practices for modern web applications, such as:
 - Regular updates to dependencies and security patches.
 - Using **Secure Coding Practices** to prevent common vulnerabilities like XSS and SQL injection.
 - Implementing secure **authentication mechanisms** and proper **session management**.

14. Retrieved from Cache

- **Remediation:**

- **Control Cache Behavior:**

- Ensure that sensitive information is not stored in the browser cache.

Add **cache-control headers** to prevent browsers from storing sensitive data in their cache. Example:

Cache-Control: no-store, no-cache, must-revalidate

-

- Ensure that pages containing sensitive information are not cached by proxies or the browser.

These remediations are designed to address each vulnerability in detail and help improve the overall security posture of the web application. Let me know if you need more information on any specific remediation!